

Section 9: UI & Developer Tools

Context

Ferrite's UI serves two purposes: basic browser chrome (tabs, navigation) and differentiating developer panels. This section covers both, plus the two standalone tools.

UI Framework: Iced

- **Architecture:** Elm-like (Model → Update → View), retained-mode
- **Renderer:** wgpu (same GPU API family as Servo/WebRender)
- **Theme:** Custom dark theme with accent colors for risk levels (green=low, amber=medium, red=high)
- **Layout:** Single window with Servo webview occupying main area, sidebar for DevTools panels (toggleable), bottom bar for status

Browser Chrome Components

- **Tab Bar:** Horizontal tabs with title, favicon, close button. Active tab highlighted. New tab button.
- **Address Bar:** URL input with navigation buttons (back, forward, reload, stop). Lock icon for HTTPS.
- **Status Bar:** Current page load status, active extension count, agent connection status, audit event counter.
- **Settings:** Modal dialog for: default policy selection, extension management, agent connection config, audit log export.

Custom DevTools Panels (sidebar)

Panel 1: Capability Inspector

- Lists all active capability tokens, grouped by principal
- Each token shows: principal, cap_type, scope, constraints, expires_at, risk_level
- Color-coded by risk level
- Click token to see: the Rego rule that granted it, the audit entry, the original request
- "Revoke" button per token
- Real-time updates as tokens are minted/expired/revoked

Panel 2: Agent Timeline

- Chronological timeline of all agent actions
- Each entry shows: timestamp, action type, target, outcome (granted/denied), risk level
- Expandable entries show: full capability request, policy evaluation details, consent decision (if any)
- Filter by: action type, outcome, risk level, time range
- "Replay" button: exports the action sequence for deterministic replay

Panel 3: Audit Log Viewer

- Searchable, filterable table of all audit events
- Columns: timestamp, event_type, principal, capability, scope, outcome
- Full-text search across all fields
- Hash chain integrity indicator (green check / red warning)
- Export button: saves filtered events as JSON for external analysis

Panel 4: AX Tree Inspector

- Displays accessibility tree of current page (extracted via ferrite-a11y)
- Tree view with expandable nodes
- Each node shows: role, name, value, states, bounding box
- Highlights which elements have been accessed by agents/extensions
- Basic WCAG violation indicators

Consent Dialog (Modal)

- Renders as a modal overlay in Iced (NOT in web content)
 - Shows: principal name, action description, target origin, risk level with explanation
 - Buttons: "Deny", "Allow Once", "Allow for Session"
 - Timeout: auto-deny after 60 seconds if no response
 - Cannot be dismissed by clicking outside — requires explicit choice
 - Visual distinction from web content: consistent border, broker icon, subtle animation
-

Standalone Tool 1: Extension Security Analyzer (CLI)

```
ferrite-analyze <extension.wasm> --manifest <ferrite-manifest.toml> --policy  
<policy-dir/>
```

What it does:

1. Loads the Wasm extension in a sandboxed Wasmtime instance (no actual browser needed)
2. Enumerates all host function imports the extension declares
3. Cross-references against the manifest's declared capabilities
4. Evaluates the manifest against the policy
5. Reports: over-permissions (declared but unused capabilities), under-declarations (attempted imports not in manifest), policy violations, resource limit assessment

Output format: JSON report + human-readable summary

Use case: Extension developers run this in CI before publishing. Security researchers use it to audit extensions.

Standalone Tool 2: Agent Action Inspector (GUI)

Standalone Iced application (separate binary from the browser):

```
ferrite-inspector <audit-log-file>
```

What it does:

1. Reads an exported audit log file (JSON lines format)
2. Verifies hash chain integrity (reports any broken links)
3. Verifies Merkle proofs (if present)
4. Displays interactive timeline identical to the browser's Agent Timeline panel
5. Allows filtering, searching, and exporting subsets
6. Generates summary report: total actions, risk distribution, denial rate, consent patterns

Use case: Post-hoc analysis of agent sessions. Forensic investigation after suspected prompt injection. Compliance reporting.

Section 10: Engine Integration (Servo + CEF)

Context

Ferrite uses two browser engines. Servo (Track A) is the primary research vehicle. CEF (Track B) is a lightweight validation harness. Both connect to the same capability broker through an engine abstraction trait.

Engine Abstraction Trait

The `ferrite-engine-trait` crate defines the interface that both Servo and CEF implement:

NavigationController:

- `navigate(url) → Result`
- `go_back() → Result`
- `go_forward() → Result`
- `reload() → Result`
- `stop() → Result`
- `current_url() → String`
- `current_title() → String`

ContentAccessor:

- `query_dom(selector, origin) → DomSubtree` (requires `dom.read` token)
- `execute_action(selector, action, value, origin) → Result` (requires `dom.write` token)
- `get_page_text(origin) → String` (requires `dom.read` token)
- `take_screenshot(region?) → Image` (requires `dom.read` token)

StateProvider:

- `get_tab_origin(tab_id) → Origin`
- `get_loading_state(tab_id) → LoadingState`
- `on_navigation(callback) →` fires when origin changes (for token revocation)
- `on_page_load(callback) →` fires when page finishes loading

AccessibilityProvider:

- `get_ax_tree(tab_id) → AXSnapshot` (requires `a11y.read` token)

Servo Integration (Track A)

Pinning Strategy

- Pin to Servo v0.0.5 (or latest stable at project start)
- One planned upgrade at month 4 with strict 1-week time-box
- If upgrade breaks >5 integration points, abort and stay on current pin

Key Integration Points

1. **WebViewDelegate trait:** Implement for navigation events, page lifecycle, input handling
2. **ServoDelegate trait:** Implement for embedding-level events (resize, focus)
3. **Request interception:** Hook into Servo's fetch pipeline to route requests through broker
4. **DOM access:** Use Servo's script thread accessors (via constellation messages) for DOM queries
5. **AX tree:** Extract from Servo's accessibility module (if available) or build from DOM

Known Servo Limitations (Accept These)

- ~79% WPT pass rate — many real-world sites will break
- Shadow DOM support incomplete
- CSS Grid partially implemented
- Service workers incomplete
- No DevTools built-in (use Firefox remote debugging)
- SpiderMonkey integration requires specific LLVM/Clang versions

CEF Integration (Track B)

Purpose

- Prove the capability broker is engine-agnostic
- Provide a web-compatible comparison target for evaluation
- Fallback if Servo embedding proves untenable

Implementation

- Use tauri-apps/cef-rs (v145+, actively maintained)
- Minimal shell: single window, basic navigation
- Same engine trait implementation as Servo
- Same broker interface — from the broker's perspective, CEF and Servo are interchangeable

Scope

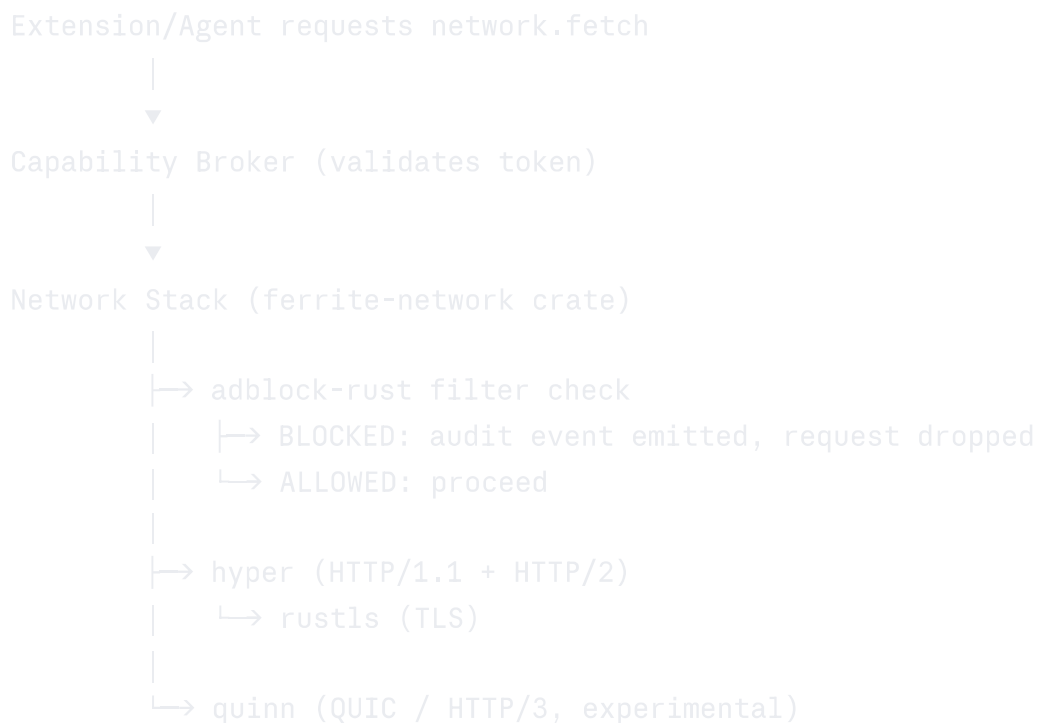
- No custom DevTools panels in CEF shell (those are Servo/Iced only)
- No polished UI — CEF harness is a testing tool, not a product
- Time budget: 2-3 weeks maximum

Section 11: Network Stack & Content Blocking

Context

The network stack handles all HTTP(S) traffic. Content blocking via adblock-rust is integrated as a first-class feature with audit logging.

Network Architecture



adblock-rust Integration

- Engine: Brave's `adblock-rust` crate
- Filter lists: EasyList + EasyPrivacy (bundled at build time, updatable)
- Integration point: before request execution, check URL + request type against filter engine
- On block: emit `RequestBlocked` audit event with: URL, filter rule that matched, estimated bytes saved
- On allow: no audit event for allowed requests (too noisy), unless extension/agent-initiated

Metrics Collected

- Total requests blocked per page load
 - Estimated bytes saved (based on content-length headers of blocked requests)
 - Third-party request ratio (blocked vs allowed)
 - These feed into the evaluation measurements
-

Section 12: Accessibility Automation

Context

The accessibility tree (AX tree) provides a semantic, structured view of page content. Ferrite exposes this to agents via the capability broker, enabling more robust automation than raw DOM access.

Why AX Tree for Agents

Raw DOM is fragile — CSS selectors break when pages change. The AX tree provides semantic meaning: "this is a button labeled 'Submit'", "this is a text field for 'Email'", "this is a navigation menu." Agents operating on the AX tree are more robust and more auditable (actions reference semantic elements, not opaque CSS selectors).

Architecture

1. **Extraction:** Servo builds an AX tree internally for assistive technology support. The `ferrite-a11y` crate hooks into this and serializes the tree as a JSON structure.
2. **Broker Gating:** AX tree access requires an `a11y.read` capability token. The token is scoped to a specific tab. Rate-limited to prevent excessive extraction (max 2

snapshots/second).

3. Serialized Format:

```
AXNode {  
  role:      String ("button", "textfield", "link", "heading", etc.)  
  name:      String (accessible name / label)  
  value:     Option<String>  
  description: Option<String>  
  states:    Vec<String> ("focused", "disabled", "expanded", etc.)  
  bounding_box: { x, y, width, height }  
  children:  Vec<AXNode>  
  element_ref: String (opaque reference for dom.write targeting)  
}
```

4. **Agent Integration:** When an agent requests `ax.snapshot`, the returned tree includes `element_ref` values that can be passed to `dom.write` for click/type actions. This provides a clean path: read AX tree → identify target element semantically → act on it.
5. **WCAG Checks:** Basic automated checks integrated into the AX Tree Inspector panel:
 - Missing alt text on images
 - Missing form labels
 - Insufficient color contrast (if computable from styles)
 - Missing heading hierarchy
 - These are informational — displayed in the panel, not enforced